

CHAPTER 04

OpenCV 주요 기능

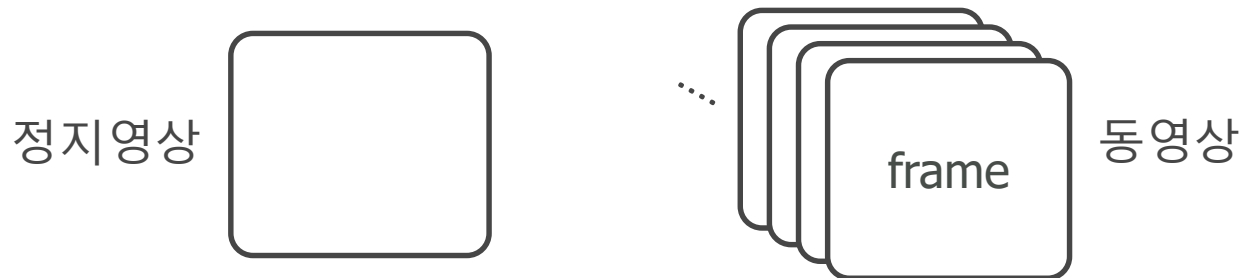
OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

4장 OpenCV 주요 기능

4.1 카메라와 동영상 파일 다루기

카메라와 동영상 파일 다루기

- 동영상이란 여러 개의 정지 영상을 압축하여 하나의 파일로 저장한 것
- 동영상에 저장되어 있는 하나의 정지 영상을 **프레임(frame)**이라고 함
- 동영상을 처리하는 작업은 동영상에서 프레임을 추출한 후, 각각의 프레임에 영상 처리 기법을 적용
- 카메라는 일정 시간 간격으로 정지 영상 프레임을 저장함
- 카메라와 동영상 파일을 다루는 작업은 연속적인 프레임 영상을 받아 와서 처리한다는 공통점이 있음
- OpenCV는 카메라 또는 동영상을 다루는 **VideoCapture클래스** 제공



VideoCapture 클래스

코드 4-1 간략화한 VideoCapture 클래스 정의

```
01  class VideoCapture
02  {
03  public:
04      VideoCapture();
05      VideoCapture(const String& filename, int apiPreference = CAP_ANY);
06      VideoCapture(int index, int apiPreference = CAP_ANY);
07      virtual ~VideoCapture();
08
09      virtual bool open(const String& filename, int apiPreference = CAP_ANY);
10      virtual bool open(int index, int apiPreference = CAP_ANY);
11      virtual bool isOpened() const;
12      virtual void release();
13
```

VideoCapture 클래스

코드 4-1 간략화한 VideoCapture 클래스 정의

```
14     virtual bool grab();
15     virtual bool retrieve(OutputArray image, int flag = 0);
16
17     virtual VideoCapture& operator >> (Mat& image);
18     virtual bool read(OutputArray image);
19
20     virtual bool set(int propId, double value);
21     virtual double get(int propId) const;
22     ...
23 };
```

- VideoCapture 클래스의 멤버 변수는 모두 protected로 선언되어 사용자가 직접 접근 안됨(opencv2/videoio.hpp)

VideoCapture 클래스: 객체 생성(동영상)

```
VideoCapture::VideoCapture(const String& filename, int apiPreference = CAP_ANY);  
bool VideoCapture::open(const String& filename, int apiPreference = CAP_ANY);
```

- `filename` 동영상 파일 이름
- `apiPreference` 사용할 비디오 캡처 API 백엔드
- 반환값 (`VideoCapture::open()` 함수) 열기가 성공하면 `true`, 실패하면 `false`

- VideoCapture 객체를 생성할 때 동영상 파일 이름을 지정함
- 기본생성자로 VideoCapture 객체를 생성할 때는 VideoCapture::open() 멤버 함수를 호출해야 함

```
VideoCapture cap("video.mp4");
```

```
VideoCapture cap;  
cap.open("video.mp4");
```

VideoCapture 클래스: 객체 생성(동영상)

- filename : *.avi, *.mpg, *.mp4 등 확장자를 갖는 동영상 파일 이름을 전달함
 - 현재 프로젝트 폴더에 동영상 파일이 있으면 "video.mp4" 형태로 파일 이름만 지정,
 - 다른 폴더에 동영상 파일이 있다면 절대 경로 또는 상대 경로를 추가하여 파일 이름을 지정함 "c:/video/video.mp4" 또는 "c:\\video\\video.mp4"
- apiPreference : 동영상 파일을 불러올 때 사용하는 API를 지정. VideoCaptureAPIs 열거형 상수 중 하나를 지정함
 - 대부분의 경우 apiPreference 인자를 생략하면 기본값인 CAP_ANY를 지정하며, 이 경우 시스템이 알아서 적절한 방법을 선택하여 사용함

VideoCapture 클래스: 객체 생성(동영상)

표 4-1 주요 VideoCaptureAPIs 열거형 상수

VideoCaptureAPIs 열거형 상수	설명
CAP_ANY	자동 선택
CAP_V4L, CAP_V4L2	V4L/V4L2(리눅스)
CAP_FIREWIRE, CAP_FIREWARE, CAP_IEEE1394	IEEE 1394 드라이버
CAP_DSHOW	다이렉트쇼(DirectShow)
CAP_PVAPI	PvAPI, Prosilica GigE SDK
CAP_OPENNI	OpenNI
CAP_MSMF	마이크로소프트 미디어 파운데이션 (Microsoft Media Foundation)
CAP_GSTREAMER	GStreamer
CAP_FFMPEG	FFMPEG 라이브러리
CAP_IMAGES	OpenCV에서 지원하는 일련의 영상 파일 (예) img_%02d.jpg
CAP_OPENCV_MJPEG	OpenCV에 내장된 MotionJPEG 코덱

VideoCapture 클래스: 객체 생성(동영상)

```
bool VideoCapture::isOpened() const;
```

• 반환값 카메라 또는 동영상 파일이 사용 가능하면 true, 그렇지 않으면 false

- VideoCapture::isOpened() 함수를 이용하여 객체 생성시 에러 체크

```
if (!cap.isOpened( )) {  
    cerr << "Video open failed!" << endl;  
    return -1;  
}
```

VideoCapture 클래스: 객체 생성(카메라)

```
VideoCapture::VideoCapture(int index, int apiPreference = CAP_ANY);  
bool VideoCapture::open(int index, int apiPreference = CAP_ANY);
```

- **index** 카메라와 장치 사용 방식 지정 번호
- **apiPreference** 사용할 카메라 캡처 API 백엔드
- **반환값** (VideoCapture::open() 함수) 열기가 성공하면 true, 실패하면 false

- 카메라 장치를 사용하는 경우에도 VideoCapture 생성자 혹은 VideoCapture::open() 멤버 함수를 사용
- **index = camera_id + domain_offset_id**
 - 1대의 카메라만 연결된 경우 camera_id -> 0
 - 2대 이상의 카메라가 연결된 경우 camera_id -> 0,1,2,...
 - domain_offset_id : apiPreference 인자값을 의미
 - apiPreference 인자가 CAP_ANY(0)인 경우 index 값은 camera_id와 같다

VideoCapture 클래스: 객체 생성(카메라)

- apiPreference : 동영상 파일을 불러올 때 사용하는 API를 지정할 수 있음. VideoCaptureAPIs 열거형 상수 중 하나를 지정함
 - 대부분의 경우 apiPreference 인자를 생략하면 기본값인 CAP_ANY를 지정하며, 이 경우 시스템이 알아서 적절한 방법을 선택하여 사용함

```
VideoCapture cap(0);  
VideoCapture cap;  
cap.open(0);
```

VideoCapture 클래스: 객체 생성(카메라)

```
bool VideoCapture::isOpened() const;
```

• 반환값 카메라 또는 동영상 파일이 사용 가능하면 true, 그렇지 않으면 false

- 카메라 열기를 수행한 후에는 VideoCapture::isOpened() 멤버 함수를 이용하여 열기 작업이 성공적으로 수행되었는지 확인하는 것이 좋음

```
if (!cap.isOpened()) {  
    cerr << "Camera open failed!" << endl;  
    return -1;  
}
```

VideoCapture 클래스: 프레임 읽기

```
VideoCapture& VideoCapture::operator >> (Mat& image);  
bool VideoCapture::read(OutputArray image);
```

- **image** 다음 비디오 프레임. 만약 더 가져올 프레임이 없다면 비어 있는 행렬로 설정됩니다.
- **반환값** 프레임을 받아 올 수 없으면 false 반환

- 카메라 또는 동영상 파일로부터 한 프레임을 받아 오기 위한 함수
- 카메라 또는 동영상 파일로부터 다음 프레임을 받아 와서 Mat 클래스 형식의 변수 image에 저장함

```
VideoCapture cap(0);  
Mat frame1, frame2;  
cap >> frame1;                    // 1st frame  
cap.read(frame2);                // 2nd frame
```

VideoCapture 클래스: 속성 얻기

```
double VideoCapture::get(int propId) const;
```

- **propId** 속성 ID. VideoCaptureProperties 열거형 중 하나를 지정합니다.
- **반환값** 지정한 속성 값. 만약 지정한 속성을 얻을 수 없으면 0을 반환합니다.

- 카메라 장치 또는 동영상 파일로부터 여러 가지 파일/장치 속성값을 받아 오기 위한 함수

```
VideoCapture cap(0);  
int w = cvRound(cap.get(CAP_PROP_FRAME_WIDTH));  
int h = cvRound(cap.get(CAP_PROP_FRAME_HEIGHT));
```

- 프레임의 폭과 높이 읽어옴, 리턴값이 실수이므로 반올림(cvRound)하여 저장

VideoCapture 클래스: 속성 얻기

- 표 4-2에 열거된 상수 중에는 동영상 파일에서만 동작하는 속성도 있고, 카메라에 대해서만 사용할 수 있는 속성도 있음

표 4-2 주요 VideoCaptureProperties 열거형 상수

VideoCaptureProperties 열거형 상수	설명
CAP_PROP_POS_MSEC	비디오 파일에서 현재 위치(밀리초 단위)
CAP_PROP_POS_FRAMES	현재 프레임 위치(0-기반)
CAP_PROP_POS_AVI_RATIO	[0, 1] 구간으로 표현한 동영상 프레임의 상대적 위치(0: 시작, 1: 끝)
CAP_PROP_FRAME_WIDTH	비디오 프레임의 가로 크기
CAP_PROP_FRAME_HEIGHT	비디오 프레임의 세로 크기

VideoCapture 클래스: 속성 얻기

표 4-2 주요 VideoCaptureProperties 열거형 상수

VideoCaptureProperties 열거형 상수	설명
CAP_PROP_FPS	초당 프레임 수
CAP_PROP_FOURCC	fourcc 코드(코덱을 표현하는 정수 값)
CAP_PROP_FRAME_COUNT	비디오 파일의 전체 프레임 수
CAP_PROP_BRIGHTNESS	(카메라에서 지원하는 경우) 밝기 조절
CAP_PROP_CONTRAST	(카메라에서 지원하는 경우) 명암비 조절
CAP_PROP_SATURATION	(카메라에서 지원하는 경우) 채도 조절
CAP_PROP_HUE	(카메라에서 지원하는 경우) 색상 조절
CAP_PROP_GAIN	(카메라에서 지원하는 경우) 감도 조절
CAP_PROP_EXPOSURE	(카메라에서 지원하는 경우) 노출 조절
CAP_PROP_ZOOM	(카메라에서 지원하는 경우) 줌 조절
CAP_PROP_FOCUS	(카메라에서 지원하는 경우) 초점 조절

VideoCapture 클래스: 속성 설정

```
bool VideoCapture::set(int propId, double value);
```

- **propId** 속성 ID. VideoCaptureProperties 열거형 중 하나를 지정합니다.
- **value** 지정할 속성 값
- **반환값** 속성 지정이 가능하면 true, 아니면 false

- 카메라 또는 비디오 파일 재생과 관련된 속성 값을 설정할 때 사용함
- 만약 video.mp4 파일을 열어서 100번째 프레임으로 이동

```
VideoCapture cap("video.mp4");  
cap.set(CAP_PROP_POS_FRAMES, 100);
```

카메라 입력 처리하기 : 객체 생성

- 방법 1: 기본 생성자를 이용하여 VideoCapture 객체 생성 후, VideoCapture::open() 멤버 함수를 이용하여 사용할 카메라 장치를 열어야 함

```
VideoCapture cap;  
cap.open(0);
```

- 방법2: VideoCapture 객체 생성과 동시에 카메라 장치를 열기

```
VideoCapture cap(0);
```

- 카메라 장치가 정상적으로 열렸는지는 VideoCapture::isOpened() 멤버 함수를 이용하여 확인해야 함

```
if (!cap.isOpened()) {  
    cerr << "Camera open failed!" << endl;  
    return -1;  
}
```

카메라 입력 처리하기 : 프레임 읽기

- 카메라의 프레임은 연산자 >>를 이용하여 Mat 클래스 객체에 저장
- Mat 객체에 저장된 영상은 imshow() 함수를 사용하여 화면에 출력

```
VideoCapture cap(0);  
if (!cap.isOpened()) {  
    cerr << "Camera open failed!" << endl;  
    return -1;  
}  
  
Mat frame;  
while (true) {  
    cap >> frame;  
  
    imshow("frame", frame);  
    waitKey(10);  
}
```

카메라 입력 처리하기 : 프레임 읽기

- 카메라에서 정지 영상 프레임을 제대로 받아 오지 못하는 경우에 대한 예외 처리도 추가하는 것이 좋음
- 프로그램 동작 중 사용자가 키보드의 특정 키를 누를 때 종료 할 수 있도록 코드를 추가하는 것이 좋음

```
Mat frame;
while (true) {
    cap >> frame;
    if (frame.empty())
        break;

    imshow("frame", frame);

    if (waitKey(10) == 27) // ESC key
        break;
}
```

코드4-2 : 영상 반전

- while 반복문 안에서 카메라로부터 프레임을 받아 왔다면 이후 각 프레임에 다양한 정지 영상 처리 기법을 적용할 수 있음
- 카메라의 매 프레임에 대하여 영상의 반전을 수행하고, 그 결과를 화면에 같이 출력할 수 있음
- 카메라 및 동영상 처리는 결국 각 프레임에 대한 정지 영상 처리와 같은 구조임을 기억하기 바람

코드4-2 : 영상 반전

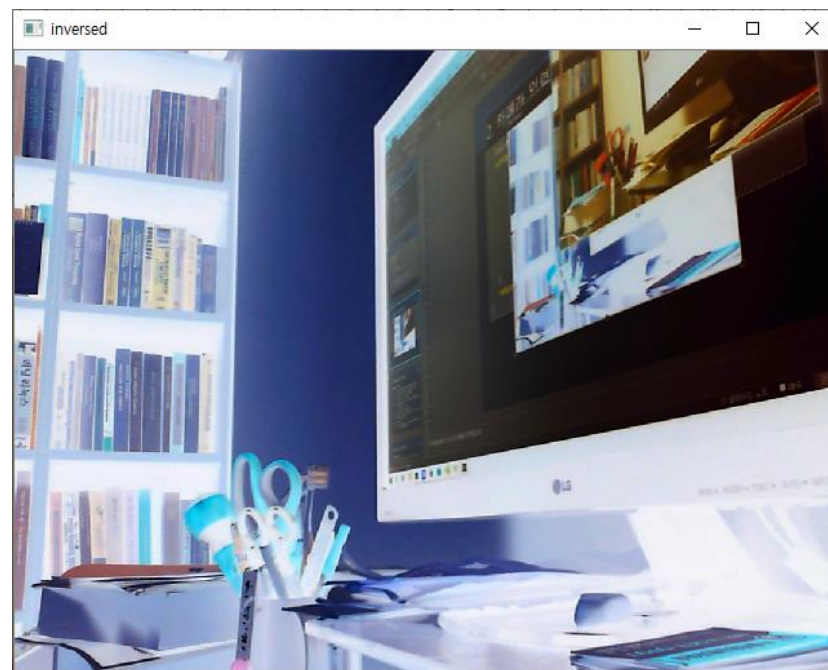
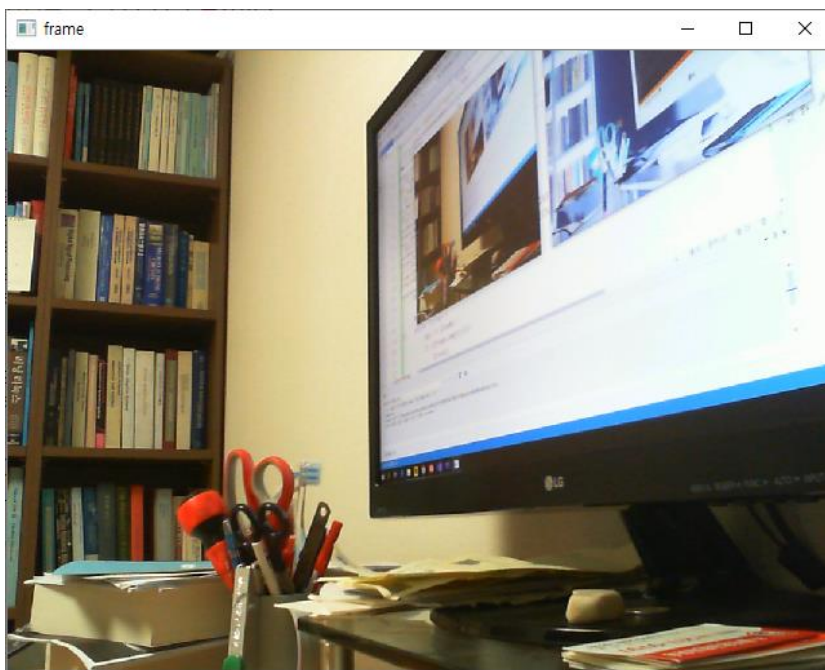
```
#include <iostream>
using namespace std;
#include <opencv2/opencv.hpp>
using namespace cv;
int main(void)
{
    VideoCapture cap(0);                                // 객체생성
    if (!cap.isOpened()) {                                // 에러처리
        cerr << "Camera open failed!" << endl;          // cerr -> cout
        return -1;
    }
    cout << "Frame width: " << cvRound(cap.get(CAP_PROP_FRAME_WIDTH))
        << endl;                                           //속성출력
    cout << "Frame height: " << cvRound(cap.get(CAP_PROP_FRAME_HEIGHT))
        << endl;                                           //속성출력
}
```

코드4-2 : 영상 반전

```
Mat frame, inversed;
while (true) {
    cap >> frame;
    if (frame.empty()){
        cerr << << "frame empty!" << endl;
        break;
    }
    inversed = ~frame;                // 비트별 not 연산
    imshow("frame", frame);
    imshow("inversed", inversed);
    if (waitKey(10) == 27)           // 27: ESC key의 아스키코드
        break;
}
return 0;
}
```

코드4-2 : 영상 반전

```
C:\coding\opencv\ch04\video\video.exe
Frame width: 640
Frame height: 480
```



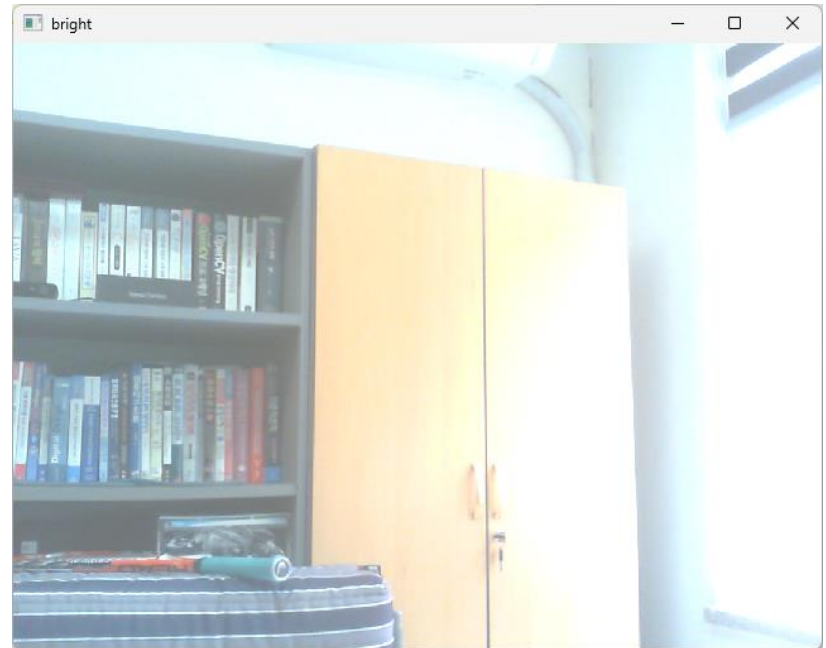
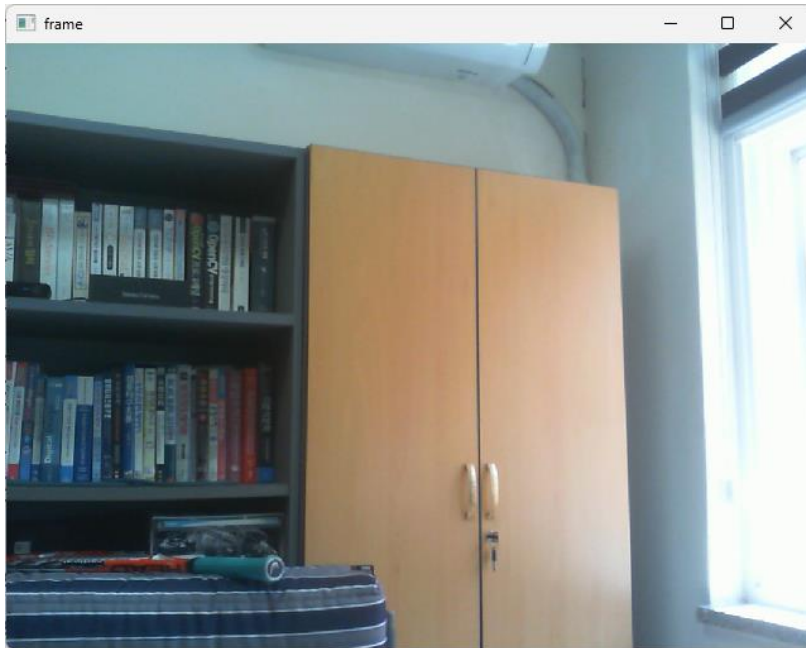
- 영상 반전: 이진수 변환 후 비트별로 not연산
Black 0(00000000) -> white 255(11111111)
white 255(11111111) -> Black 0(00000000)

실습과제 1

- 다음 용어에 대하여 조사하시오.
 - API(Application Programming Interface)
 - V4L
 - FFMPEG
 - DirectShow
 - MSMF
 - GStreamer

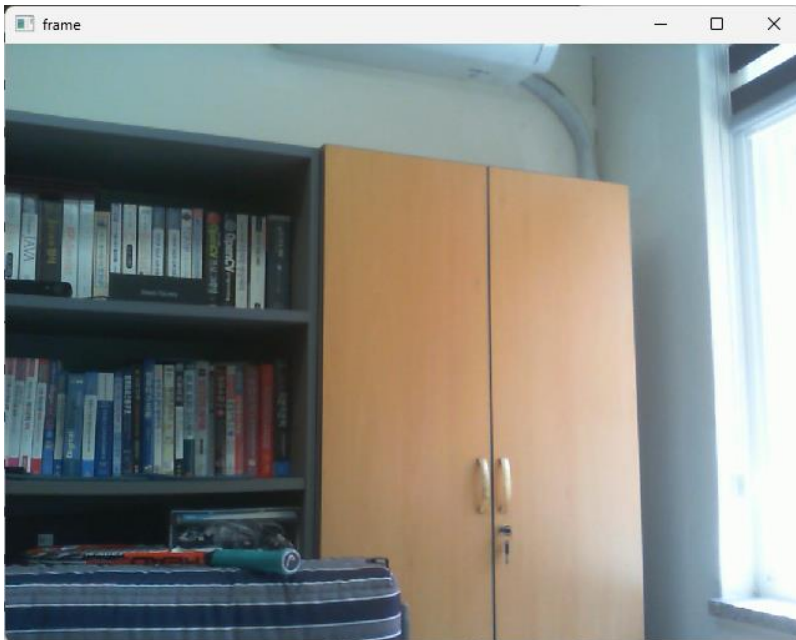
실습과제2

- 카메라로 촬영한 컬러영상의 R,G,B값 모두 100만큼 증가시킨 후 원본과 결과영상을 모두 출력하라.



실습과제3

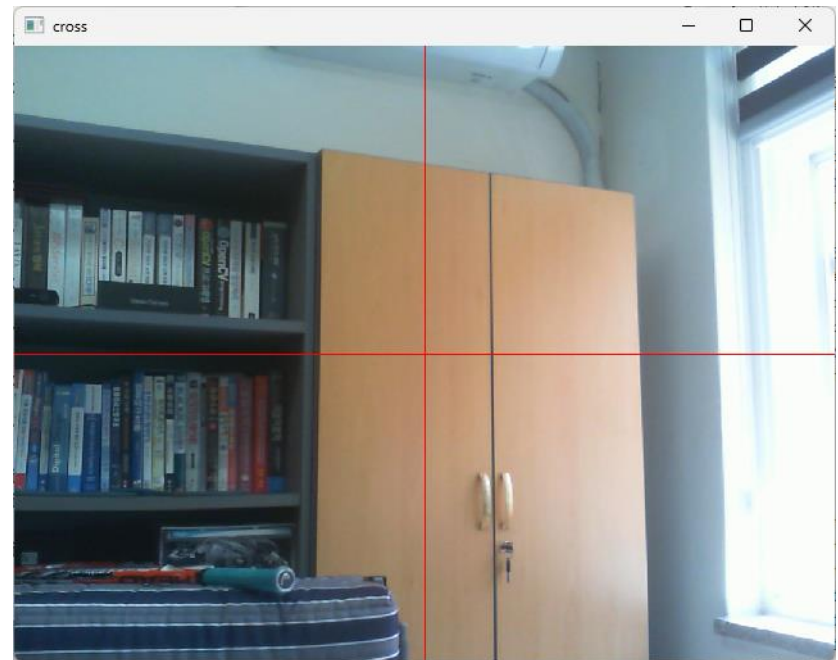
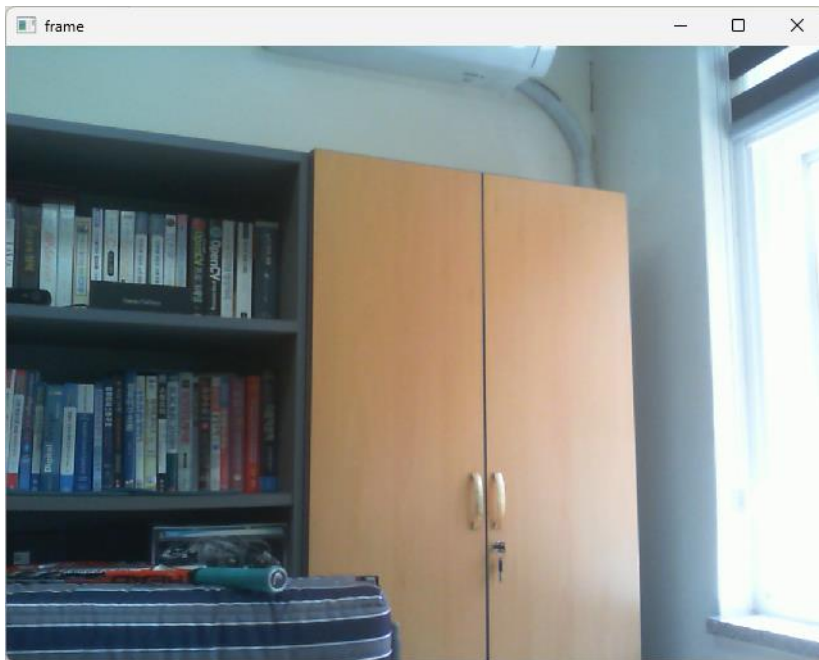
- 카메라로 촬영한 영상을 화면에 출력하라.
- 무한루프로 작성하고 Q 또는 q를 누르면 종료하도록 하라.
- s를 누를때마다 현재 프레임을 파일로 저장하라. 파일명은 frame00.jpg 형식으로 누를 때마다 숫자는 증가하도록 하라 -> 규칙적인 파일명을 만들기 위해 3장에서 배운 format 함수를 사용



frame00.jpg	↺	2025-04-07 오후 12:56	JPG 파일
frame01.jpg	↺	2025-04-07 오후 12:56	JPG 파일
frame02.jpg	↺	2025-04-07 오후 12:56	JPG 파일
frame03.jpg	↺	2025-04-07 오후 12:56	JPG 파일
frame04.jpg	↺	2025-04-07 오후 12:56	JPG 파일

실습과제4

- 카메라로 촬영한 영상을 화면에 출력하라.
- 무한루프로 작성하고 q를 누르면 종료하라.
- 새로운 창을 출력하고 원본에 빨간색으로 다음처럼 그린 후 출력하라 -> 원본과 처리결과를 저장하는 Mat 객체를 분리하여 저장하라.



과제 제출 방법

- 소스파일, 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출 하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.